

PowerMCP 实践指南

让 LLM 真正操作电力仿真软件

仓库: [Power-Agent/PowerMCP](#) | **许可:** MIT | **Stars:** 217 | **代码:** 约 26,000 行 Python **定位:** 电力系统软件的 MCP 服务器集合, 把 PSS/E、PowerWorld、OpenDSS、pandapower 等 15 款软件的 API 封装成 LLM 可直接调用的工具 **适用读者:** 电网工程师、新能源开发商技术团队、电力 AI 应用开发者

一、这个项目到底解决什么问题

1.1 电力 AI 最硬的那道墙

假设你让一个通用大模型回答: "把 IEEE 39 节点系统的 L7 线路断开后, 哪条线路会过载?"

它可能会给你一个听起来很专业的答案。但你无法验证它是真的算过, 还是编的。

原因很直接: **电力行业的核心仿真工具 (PSS/E、PowerWorld、PowerFactory、PSCAD、PSLF) 几乎全部是闭源商业软件**, LLM 无法访问、无法调用、无法验证。模型再聪明, 也只能停留在"文本层面", 无法触及"物理真实"。

PowerMCP 的解法很朴素但很彻底: **为每一款电力软件写一个 MCP 服务器, 把软件的底层 API 变成 LLM 可以直接调用的工具。**

用户提问

↓

LLM (Claude / GPT / 本地模型)

↓ MCP 协议

PowerMCP 服务器 (15 个)

↓ 调用厂商 API

PSS/E · PowerWorld · OpenDSS · pandapower · PyPSA · ANDES ...

↓ 返回真实计算结果

LLM 基于物理结果作答 ← 而不是基于想象

MCP 是什么: Anthropic 提出的 Model Context Protocol (模型上下文协议), 作用是给 AI 应用提供统一的工具连接标准。你可以把它理解为"AI 世界的 USB-C 接口"——不论什么模型、什么软件, 都通过同一个协议对接。

1.2 一句话总结价值

PowerMCP 的价值不在于让 LLM 更聪明, 而在于让 LLM 的结论第一次变得"可验证"。

二、工具矩阵: 15 款软件怎么选

这是实践中最常被问到的问题。先看清楚全景, 再决定装什么。

2.1 完整清单

工具	类型	软件领域	主要用途	安装方式
pandapower	开源	稳态 AC 分析	潮流、N-1 分析、网络建模	核心内置
PyPSA	开源	规划与运行优化	OPF、投资优化、容量扩张	核心内置
PowerIO	开源	格式转换	跨软件案例翻译、矩阵构建	核心内置
OpenDSS	开源	配电网分析	馈线潮流、日能量、谐波	[opendss]
ANDES	开源	动态安全/小信号	潮流、特征值、时域仿真	[andes]
Egret	开源	市场与运行优化	DC-OPF、AC-OPF、机组组合	[egret]
surge	开源	输电分析全流程	PTDF/LODF、OPF、ATC、SCED/SCUC	[surge]
LTSpice	免费	电路仿真	网表生成、无头仿真、波形	[ltspice]
GenX	开源 (Julia)	容量规划	容量结果聚合、分区绘图	[genx]
HOPE	开源 (Julia)	电力系统优化	本地/远程双入口	[hope]
PowerWorld	商业	输电网稳态分析	潮流、N-1、PTDF/LODF、Jacobian	[powerworld]
PSS/E	商业	输电网潮流与动态	潮流求解 + 全量 PSSPY 命令桥接	[psse] + 路径配置
PSLF	商业	GE 潮流与稳定	拓扑编辑、N-1 分析	[pslf] + 路径配置
PowerFactory	商业	机电暂态/RMS	RMS 仿真、潮流、短路、结果导出	[powerfactory] + 路径配置
PSCAD	商业	电磁暂态仿真	项目管理、仿真自动化	[pscad-windows]
PLEXOSDB	商业 (免许可)	电力市场建模	29 个 CRUD 工具 + Sienna 转换	[plexosdb] (需手动装)

2.2 选型建议（按团队类型）

你的情况	推荐组合	理由
初次试水 / 教学演示	pandapower + PyPSA + PowerIO	零成本、零配置、pip 即装即用
高校科研	上述 + surge + ANDES + Egret	覆盖输电全流程 + 动态分析 + 优化
新能源并网研究	pandapower + OpenDSS + PyPSA	配电网 + 稳态 + 规划
电网公司调度侧	powerworld 或 psse + PowerIO	匹配现有生产工具链
电力市场分析	surge + Egret + PLEXOSDB	市场出清与调度
电磁暂态研究	pscad + PowerFactory + LTSpice	需要商业许可与 Windows

务实提醒：如果你还不确定要不要投入商业软件许可，先用纯开源组合跑通一个完整案例。开源的 pandapower + PyPSA + surge 已经能覆盖相当大比例的日常分析需求，验证价值后再决定是否扩充。

三、安装与配置（实操）

3.1 最简安装路径

```
# 1) 安装核心 (含 pandapower + PyPSA + PowerIO, 无需任何配置)
pip install powermcp

# 2) 运行交互式向导
powermcp install
```

向导会依次完成三件事：

1. **让你勾选工具**（↑↓ 移动，**空格键切换选中**，回车确认）
2. **采集闭源软件的本地安装路径**（如 PSS/E、PSLF 的 Python 库路径）
3. **自动写入 MCP 客户端配置**（支持 Claude Desktop、Claude Code、Codex CLI 三类）

注意：在勾选界面直接按回车，只会保留预选项（pandapower + PyPSA + PowerIO）。一定要用**空格**来增减选择。

3.2 命令行直选（不想用勾选界面）

```
powermcp install --tools psse,andes      # 核心 + 指定工具
powermcp install --all                   # 当前平台所有可用工具
powermcp install --dry-run               # 只预览改动，不实际写入
powermcp install --yes                   # 非交互，只装核心
```

一个重要特性：重新运行 `powermcp install` 时，**上次已安装或已配置的工具会被自动预勾选**，所以它是"更新"而不是"重置"。这对逐步扩充工具集的团队很友好。

3.3 安装衍生工具包（可选）

```
pip install powermcp[psse]              # PSS/E 支持
pip install powermcp[andes,opendss]     # 一次装多个
pip install powermcp[opensource]        # 全部开源工具
pip install powermcp[all]                # 全部（闭源软件仍需本地安装）
```

3.4 闭源工具的路径配置

这些工具包装的是本地安装的商业软件，PowerMCP 需要知道它们在哪。配置存储在 `~/.powermcp/config.toml`。

工具	配置键	示例
PSS/E	<code>psse.python_lib</code> 、 <code>psse.bin</code>	<code>powermcp config set psse.python_lib "C:\Program Files\PTI\PSSE36\36.2\PSSPY311"</code>
PSLF	<code>pslf.python_lib</code>	<code>powermcp config set pslf.python_lib "C:\Program Files\GE PSLF\PSLF_PYTHON"</code>
PowerFactory	<code>powerfactory.python_path</code>	<code>powermcp config set powerfactory.python_path "... \DIgSILENT\PowerFactory 2024\Python\3.11"</code>
LTSpice	<code>ltspice.exe</code> (自动探测)	通常在标准位置自动找到，无需手动配置
HOPE	<code>hope.repo_root</code> 、 <code>hope.julia_bin</code>	<code>powermcp config set hope.repo_root "C:\src\HOPE"</code>
GenX	<code>genx.repo_root</code>	<code>powermcp config set genx.repo_root "/home/me/GenX.jl"</code>
PowerWorld	无需配置	<code>esa</code> 通过 COM 自动发现正在运行且有许可的 Simulator
PSCAD	无需配置	由 <code>pip install powermcp[pscad-windows]</code> 提供 <code>mhi-pscad</code>

3.5 验证安装

```
powermcp doctor      # 逐个检查工具的依赖与路径配置
powermcp list        # 列出可用工具、extras、平台限制
```

`doctor` 是排查问题的第一站。如果某个工具报错，先看它，再决定是装依赖还是配路径。

3.6 生成的客户端配置长什么样

```
{
  "mcpServers": {
    "powermcp_pandapower": { "command": "powermcp", "args": ["run", "pandapower"] },
    "powermcp_pypsa":      { "command": "powermcp", "args": ["run", "pypsa"] },
    "powermcp_powerio":    { "command": "powermcp", "args": ["run", "powerio"] },
    "powermcp_psse":       { "command": "powermcp", "args": ["run", "psse"] }
  }
}
```

工作区陷阱（值得单独提醒）：有用户报告 Windows 上的 **Codex Desktop 应用会覆盖** `~/.codex/config.toml`。如果你同时使用 Desktop 和 CLI，Desktop 改过配置后需要**重新运行一次** `powermcp install`。Codex CLI 本身不受影响。

3.7 不用安装，直接从克隆运行

每个服务器都是独立脚本，克隆仓库后可直接跑：

```
python pandapower/panda_mcp.py
python PSSE/psse_mcp.py      # 会读取 ~/.powermcp/config.toml
python -m powerio.mcp        # powerio 自带服务器
```

四、各工具能力速查

4.1 PowerWorld（输电网稳态分析）

最完整的输电网稳态工具集，共 14 个工具：

工具	作用
<code>open_case(case_path)</code>	打开案例文件，返回基本信息
<code>run_powerflow(solution_method='RECTNEWT')</code>	求解潮流，返回越限与电压违规
<code>analyze_contingencies(option="N-1", validate=False)</code>	N-1/N-2 故障分析
<code>get_power_flow_results(object_type, additional_fields)</code>	取指定对象类型（bus/gen/load/shunt/branch）结果
<code>get_key_field_list(object_type)</code>	查某对象类型有哪些可用字段
<code>change_parameters_multiple_element(...)</code>	批量修改同类型元件参数
<code>change_and_confirm_params(...)</code>	修改参数并 验证是否生效
<code>get_ybus(full=False)</code>	母线导纳矩阵（完整/稀疏）
<code>get_jacobian(full=False)</code>	潮流雅可比矩阵
<code>get_lodf_matrix(precision=3, method='DC')</code>	线路开断分布因子矩阵
<code>get_ptdf_matrix_fast()</code>	功率转移分布因子矩阵（快速算法）
<code>to_graph(node='bus', geographic=False)</code>	转成 NetworkX 图
<code>determine_shortest_path(start, end, ...)</code>	两母线间最短路
<code>run_robustness_analysis()</code>	系统鲁棒性分析

实践提示： `change_and_confirm_params` 比 `change_parameters_multiple_element` 更值得优先使用——它会**确认改动真的生效了**，避免“以为改了但其实没改”的静默失败。这在自动化流程里非常关键。

4.2 PSS/E（含独门绝技：命令动态桥接）

PSS/E 有一个别的工具都没有的能力——**不只是暴露固定工具，还让模型自己能查、能调 PSS/E 的底层命令：**

层级	工具	说明
基础	<code>open_case(case)</code>	打开案例
基础	<code>solve_case()</code>	求解潮流
发现	<code>search_psspy_commands(query, category)</code>	按关键词搜索 PSSPY 命令
发现	<code>lookup_psspy_command(function_name)</code>	查某个命令的完整签名
执行	<code>run_psspy_command(function_name, arguments)</code>	执行任意 PSSPY 命令

这是 PowerMCP 里最重要的设计选择之一。 别的工具只能做开发者预设的那几十件事；PSS/E 这条链路等于把 PSS/E 数千个 API 全部开放给模型。代价是需要模型先搜索、再查签名、最后执行——但能力上限完全不同。

使用纪律（来自官方技能文档）：- **不要把 `run_psspy_command` 当搜索工具用**——先 `search` 或 `lookup` - 每条执行的命令都要绑定一个研究目标，而不是"试试这个 API"

4.3 surge（开源输电网全流程，44 个工具）

如果不想买商业许可，surge 是目前覆盖最全的开源选择：

案例 I/O → 潮流(AC/DC/FDPF) → 灵敏度(PTDF/LODF/OTDF)
→ OPF(DC/AC/SCOPF) → N-1/N-2 故障分析
→ NERC/AC 可用输电能力 → SCED/SCUC 调度
→ 检查/导出/图分析 → 网络构建与编辑

系统依赖（容易踩坑，先装好）：

平台	命令
Ubuntu / Debian	<code>sudo apt install libhighs-dev coinor-libipopt-dev</code>
macOS	<code>brew install highs ipopt</code>

关键提醒：`pip install highspy` 单独安装不满足 HiGHS 的运行时链接要求，必须装系统库。库在非标准路径时，设置 `HIGHS_LIB_DIR` 和 `IPOPT_LIB_DIR`。

Gurobi、COPT、CPLEX 是运行时插件——**有许可就自动检测使用**，没有则回退到 HiGHS/Ipopt。

4.4 ANDES（动态分析）

工具	作用
<code>run_power_flow(file_path, dyr_path=None)</code>	潮流求解，可选附加 <code>.dyr</code> 动态模型文件
<code>run_time_domain_simulation(step_size=0.01, t_end=10.0)</code>	时域仿真
<code>run_eigenvalue_analysis(file_path)</code>	特征值（小信号）分析
<code>get_system_info()</code>	系统信息

`run_power_flow` 返回的两个关键字段（很容易被忽略，但很重要）：

- `n_dynamic_generators`：实际加载的动态发电机模型数量（汇总 ANDES 的 `SynGen`、`RenGen`、`DG` 三组）
- `dynamic_models_loaded`：布尔值

这是一个设计得很诚实的细节：`dynamic_models_loaded` 是根据实际加载了什么推导出来的，而不是根据你是否传了 `dyr_path`。原因是 ANDES 的 PSS/E dyr 解析器会跳过它不支持的模型类型——所以即使你传了 `.dyr`，系统仍可能没有任何动态模型。

反过来说，像 `kundur_full.json` 这种自身内嵌动态的案例，不传 `.dyr` 也会报告 `True`。

实践含义：想跑时域仿真或特征值分析前，**必须先确认 `dynamic_models_loaded` 为 `True`**，否则你算的是一个没有动态的静态拓扑。

4.5 PyPSA（规划与优化）

工具	作用
<code>load_network(file_path) / create_network(...)</code>	加载或新建网络
<code>get_network_info(network_name)</code>	网络统计
<code>get_component_details(...)</code>	元件详情
<code>run_power_flow(network_name, linear=False)</code>	潮流（可行性检查）
<code>optimize_network(...)</code>	运行/调度优化
<code>optimize_investment(...)</code>	容量扩张优化
<code>add_bus / add_generator / add_load / add_line / add_storage_unit</code>	网络构建
<code>import_from_csv_folder / export_to_csv_folder</code>	CSV 导入导出

环境要求：PyPSA $\geq 0.35.2$ 且 < 2 ；至少一个 LP 求解器（推荐开源 HiGHS；商业可用 Gurobi 学术版）。

4.6 pandapower（稳态 AC）

工具	作用
<code>load_network(file_path) / create_empty_network()</code>	加载/新建
<code>get_network_info()</code>	统计与数据
<code>run_power_flow(algorithm, ...)</code>	潮流（Newton-Raphson / 前推回代）
<code>run_contingency_analysis(contingency_type, elements)</code>	N-1/N-2 故障分析
<code>load_network_from_any / load_network_from_json</code>	通过 PowerIO 导入
<code>export_network_to_format(to_format)</code>	通过 PowerIO 导出

4.7 OpenDSS（配电网）

工具	作用
<code>compile_and_solve(dss_file)</code>	编译并求解馈线
<code>get_total_power() / get_bus_voltages()</code>	基态功率与电压
<code>set_load_multiplier(load_mult)</code>	负荷倍率扫描
<code>run_daily_energy_meter(meter_name, hours)</code>	日能量计量
<code>get_harmonic_results(load_name, harmonic)</code>	谐波结果

还附带 `Plotly` 可视化能力：`get_voltage_profile_plotly_figure`（电压分布图）、`get_opendss_circuit_map_plotly_figure`（电路地图）。

4.8 Egret（市场与优化）

三层递进，从最便宜能回答问题的模型开始：

```

solve_dc_opf()                ← 先做，快速可行性与阻塞筛查
    ↓ 需要电压/无功真实性时
solve_ac_opf()
    ↓ 需要时段、备用、启停逻辑时
solve_unit_commitment_problem()

```

4.9 其他工具

工具	关键能力
PowerFactory	<code>connect</code> 、 <code>activate_study_case</code> 、 <code>run_loadflow</code> 、 <code>run_rms_simulation</code> 、 <code>export_results_to_csv</code> 、 <code>export_project_to_pfd</code>
PSCAD	基于 AST 解析 <code>mhi-pscad</code> 库的装饰器与类型提示， 给 AI 深度 API 感知 ；含 30 秒执行看门狗防止挂死
LTSpice	网表生成、无头仿真、波形选择与绘制
GenX	<code>plot_capacity</code> （按类型聚合绘图）、 <code>check_capacity_setting</code> （brownfield/greenfield 识别）、 <code>summarize_capacity</code>
HOPE	双入口：本地 stdio 全权限 + 远程只读 HTTP （供 ChatGPT 网页开发者模式）
PLEXOSDB	29 个对象/属性/成员 CRUD + <code>translate_to_sienna</code> + <code>compare_solutions</code> ； 无需 PLEXOS 许可或安装

五、核心机制：PowerIO 跨软件案例翻译

这是 PowerMCP 工程上最有价值的设计，值得单独一章。

5.1 问题：每种软件说自己的"语言"

`.raw`（PSS/E）、`.pwb` / `.aux`（PowerWorld）、`.dss`（OpenDSS）、`.nc`（PyPSA）、`.m`（MATPOWER）.....格式各不相同。过去要跨软件分析，只能靠人工导出导入。

5.2 解法：一个统一的 JSON 传输格式

PowerIO 作为**核心依赖**（不是可选组件）内置于 PowerMCP，承担"翻译层"职责：

```

parse(path="case9.raw")          # 解析成规范 JSON 传输格式
    ↓ 返回 {"json": ..., "summary": ...}

load_network_from_json(network_json=...)    # → pandapower
load_model_from_json(network_json=...)      # → egret
import_case_from_json(..., output_path="case9.nc") # → PyPSA
matrix(kind="ptdf", json=...)              # → 直接建矩阵
save(to_format="psse", out_path="case9.raw", json=...) # → 落盘给只认路径的软件

```

核心思路：解析一次，在工具调用之间传递返回的 `json` 字符串，只在后端确实需要文件时才落盘。

5.3 能构建的矩阵

PowerIO 可直接构建求解器需要的稀疏矩阵：

矩阵	用途
B'、B''	快速解耦潮流
Y_bus	节点导纳矩阵
PTDF	功率转移分布因子
LODF	线路开断分布因子
Laplacian	图拉普拉斯矩阵
LACPF	线性 AC 潮流

5.4 包格式（.pio.json）：带状态的案例

除了纯 JSON 传输，PowerIO 还支持**包格式**，它能携带：

- 模型 + 包元数据
- 结构化诊断信息（`diagnostics(package_json=pkg)`）
- **来源与源码映射**（provenance / source maps）
- 稳定的行标识
- 校验状态
- **运行点序列**（operating point series）
- **累积研究提交**（cumulative study commits）
- 降维历史（lowering history）

5.5 一个体现工程严谨性的设计：拒绝静默求解

这是 PowerMCP 里最值得称赞的一个安全设计。

场景：包内存储了多个运行状态，但调用方没有指定用哪个。

```
# 静态包可以直接加载
import_case_from_json(network_json=pkg, output_path="case9.nc")

# 含多个存储状态的包，必须显式选择
import_case_from_json(
    network_json=pkg,
    output_path="dispatch.nc",
    operating_point=3      # ← 必须显式指定
)

load_network_from_json(network_json=pkg, study_commit=1)  # pandapower
```

如果调用方不选，PowerMCP 会报错，而不是默认求解基础模型。

同样的保护还有两处：

保护机制	行为	防止的问题
未选状态拒绝	含多状态包未指定 → 报错	悄悄求解错的运行点
多导体包拒绝	平衡求解器拒绝多相包，除非显式降维	配电网到输电网的有损归约被隐式执行
路径预检	显式与旧式派生的 CSV 写入目标都做预检	越界写文件

为什么这很重要：在电力工程里，一个悄悄算错的结果比一个明确的报错危险得多。报错你可以修；算错了你可能直接拿去做决策。PowerMCP 选择了"宁可报错也不猜"——这是一个成熟的工程设计取向。

5.6 路径沙箱

PowerIO 的 MCP 工具只接受**本地路径**和 `file://` **URI**，非本地 URI 方案一律拒绝。可通过环境变量限定可访问目录：

```
export POWERIO_MCP_ALLOWED_ROOTS="/data/cases:/home/user/projects"
```

诚实的技术说明（官方文档中明确写出）：这些是**预检（preflight）检查**，不是完全隔离。另一个进程理论上可以在检查之后、后端打开文件之前替换掉那个条目。这是"降低风险"而非"消除风险"。

5.7 特殊案例：PowerWorld 显示文件

PowerWorld 的 `.pwd` 显示文件需要单独解码：

```
display(path="...") # 返回图纸画布 + 每个变电站的显示坐标
```

注意：**显示几何信息与 `.pwb` / `.aux` 的案例数据是两回事**，不要混淆。

六、实战示例

示例 1：完整的 N-1 安全分析（开源工具链）

目标：分析 IEEE 39 节点系统的 N-1 安全水平。

准备：

```
pip install powermcp
powermcp install          # 勾选 pandapower + PyPSA + PowerIO
powermcp doctor           # 确认环境正常
```

对话（Claude Desktop / Claude Code / Cursor）：

你：加载 case39.json，先跑基态潮流，然后做 N-1 分析，把过载元件和电压越限母线都列出来。

模型：调用 `load_network("case39.json")`
→ 调用 `get_network_info()`
返回：39 母线、34 线路、12 变压器、10 发电机
→ 调用 `run_power_flow()`
返回：收敛
→ 调用 `run_contingency_analysis()`
返回：N-1 排序结果

典型输出（基于公开基准数据）：

【基态潮流】收敛

线路过载: L17 = 105.5%、L16 = 103.3% (限值 100%)

电压越限: 母线 7 = 0.9473 pu、母线 6 = 0.9479 pu (下限 0.95 pu)

【N-1 分析】

L7 开断 → L6 负载率 119.3%，母线 7 降至 0.9396 pu

L12 开断 → L8 负载率 119.3%、L17 111.1%，母线 7 降至 0.9379 pu

L20 开断 → L4 负载率 116.2%

关键观察：三个故障都指向**同一个薄弱区域**（母线 7 附近），这提示可以用**一个统一措施**覆盖整族故障，而不是逐个处理。这正是后续 PowerSkills 技能里“按共同走廊/断面分组”的思路。

示例 2：用 PSS/E 的命令桥接做自定义分析

场景：你需要执行一个 PSS/E 标准工具没覆盖的操作。

你：我想在 PSS/E 里改一下 G3 的机端电压目标值，
先查一下应该用哪个 PSSPY 命令。

模型：调用 `search_psspy_commands(query="voltage setpoint")`

→ 返回候选命令列表，例如 `change` 相关命令

调用 `lookup_psspy_command(function_name="...")`

→ 返回完整签名、参数个数与类型、错误码说明

调用 `run_psspy_command(function_name="...", arguments=[...])`

→ 执行

调用 `solve_case()`

→ 重新求解，确认效果

注意这个流程的三段式：搜索 → 查签名 → 执行。**不要跳过前两步直接执行**——PSSPY 命令的参数顺序和数据类型很严格，猜错会得到无意义的错误码。

示例 3：跨软件案例翻译

场景：你有一个 PSS/E 的 `.raw` 案例，但想用 pandapower 快速筛查。

你：解析 `case9.raw`，然后导入 pandapower 跑潮流。

模型：调用 `parse(path="case9.raw")`

→ 返回 `{"json": "...", "summary": {...}}`

`summary.elements.buses = 9`

`summary.elements.branches = 9`

`summary.topology.connected_components = 1`

调用 `load_network_from_json(network_json=<上面的 json>)`

调用 `run_power_flow()`

同一个 json 可以继续给别的工具用：

```
调用 import_case_from_json(network_json=<同一个 json>,
                           output_path="case9.nc") # → PyPSA
调用 matrix(kind="ptdf", json=<同一个 json>)      # → 建 PTDF
调用 save(to_format="psse", out_path="out.raw",
          json=<同一个 json>)                     # → 落盘
```

效率提示：解析一次，多处复用。不要为每个工具重新 `parse`。

示例 4：动态稳定性筛查（ANDES）

场景：检查系统的振荡模式阻尼是否达标。

你：对 IEEE 14 节点案例做特征值分析，看看有没有阻尼不足的模式。

模型：调用 `run_power_flow("ieee14.xlsx")`

- 基态求解完成，14 母线、5 发电机

调用 `get_system_info()`

- 确认动态模型、励磁与调速器类型

调用 `run_eigenvalue_analysis("ieee14.xlsx")`

- 返回 `modes` 列表（已按阻尼最差优先排序）

输出解读：

主导模式：频率 0.42 Hz，阻尼比 3.1%

工程判据：阻尼比 **低于约 5%** 通常视为需要关注。3.1% 说明这个案例的阻尼不足，应该转给动态稳定性处置流程。

重要提示： `run_eigenvalue_analysis` 返回的 `modes` 列表是**重新排序过的**（按最差阻尼优先），但 `participation_factors` 矩阵**没有重排**，仍是 ANDES 原生顺序。查参与因子时**要用 mode 的 `index` 字段，而不是它在 `modes` 列表里的位置**。这是一个容易踩的坑。

示例 5：配电网 DER 接入筛查（OpenDSS）

你：编译 `feeder.dss`，跑一下潮流，看看电压分布。

模型：调用 `compile_opendss_file(dss_file="feeder.dss")`

调用 `get_total_power()`

调用 `get_bus_voltages()`

- 返回各母线电压，识别薄弱点

调用 `get_voltage_profile_plotly_figure()`

- 生成电压分布可视化

扩展场景（负荷增长筛查）：

```
调用 set_load_multiplier(load_mult=1.25) # 负荷增长 25%
重新求解 → 对比电压变化
```

示例 6：从案例文件保存为其他格式

场景：OpenDSS 需要 `.dss` 文件，但你手上是别的格式。

```
调用 save(to_format="dss", out_path="feeder.dss",
          json=..., json_format="bmopf-json")
调用 compile_opendss_file(dss_file="feeder.dss")
```

七、已知限制与注意事项

7.1 环境限制

限制	说明	应对
商业许可	PSS/E、PowerWorld、PowerFactory、PSCAD 均需有效许可	先用开源组合验证价值
Windows 绑定	PowerWorld 走 COM 自动化；PSCAD 依赖 <code>mhi-pscad</code> （仅 Windows）	非 Windows 环境选开源方案
Python 版本冲突	surge 需 3.12–3.14；PSS/E 需 numpy 1.x	用独立虚拟环境隔离
PyPSA 版本	需 $\geq 0.35.2$ 且 < 2	注意不要装到 2.x
求解器系统库	surge 需 HiGHS/Ipopt 系统库， <code>pip install highspy</code> 不够	见 4.3 节安装表

7.2 明确的功能边界

文档中坦诚标注的不足：

- **ANDES 服务器**："under active development and may need further modification"（开发中，可能仍需调整以处理某些内部输出并确保完整兼容）
- **GenX**："These functionalities are for after a GenX run has been manually configured"（**仅适用于手动配置好的 GenX 运行之后**）—— 它不做求解，只做结果后处理
- **PLEXOSDB**：`plexosdb-mcp` 工具层**不在 PyPI 上**，需按各目录 README 手动安装
- **surge**：需要 Python 3.12–3.14（pyo3/maturin 的版本范围限制）

7.3 实践避坑清单

坑	现象	正确做法
安装向导	直接回车只装核心工具	用 空格键 勾选
ANDES 动态	传了 <code>.dyr</code> 但时域仿真没有动态	检查 <code>dynamic_models_loaded</code> 是否为 True
PSS/E 命令	直接 <code>run_psspy_command</code> 报参数错误	先 <code>search</code> → 再 <code>lookup</code> → 最后 <code>run</code>
特征值参与因子	用列表位置索引结果不对	用 mode 的 <code>index</code> 字段
surge 求解器	只 <code>pip install highspy</code> 后仍失败	装系统库 <code>libhighs-dev</code> / <code>coinor-libipopt-dev</code>
Codex 配置	Desktop 应用后配置被重置	重新运行 <code>powermcp install</code>
PowerIO 路径	非本地 URI 被拒绝	只用本地路径或 <code>file://</code>
多状态包	不指定运行点直接报错	显式传 <code>operating_point</code> 或 <code>study_commit</code>

八、能力边界：PowerMCP 不做什么

理解边界和理解能力同样重要：

PowerMCP 是	PowerMCP 不是
工具接入层——让 LLM 能调用软件	❑决策层——不会自己判断该做什么分析
暴露各软件的原始能力	❑不替代专业的电力分析软件
提供标准化的调用协议	❑不做工作流编排（那是 PowerWF 的事）
提供跨格式翻译（PowerIO）	❑不做领域知识与处置经验（那是 PowerSkills 的事）

关键认知：PowerMCP 给了智能体"手"，但没有给"经验"。一个拥有全部工具的智能体，可能一上手就跑 N-2 分析，而实际上**基态潮流都还没收敛**——这是初级工程师的典型错误。

解决这个问题的是 **PowerSkills**（技能层，教智能体"先做什么、验证什么、何时升级"）和 **PowerWF**（编排层，把工具串成业务流程）。三者配合才构成完整的解决方案。

建议的搭配顺序：**PowerMCP（能调用）→ PowerSkills（知道怎么做）→ PowerWF（成流程）**。

九、快速上手清单

```
# ① 最小可用安装
pip install powermcp
powermcp install          # 空格勾选工具
powermcp doctor           # 验证

# ② 在 MCP 客户端里确认服务器已连接
# Claude Desktop / Claude Code / Cursor 都应能看到 pandapower、pypsa、powerio

# ③ 第一个测试对话
# "加载 case39.json，跑潮流，报告母线电压范围和最重载线路"
```

建议的第一个完整练习：用示例 1（IEEE 39 节点 N-1 分析）走通全流程。它同时验证了案例加载、潮流求解、故障分析三条链路，而且输出结果有公开基准可以对照，方便判断是否配置正确。

扩展方向： - 想研究规划优化 → 加 `PyPSA`，试 `optimize_network` / `optimize_investment` - 想做动态分析 → 加 `ANDES`，注意先确认 `dynamic_models_loaded` - 想覆盖全流程 → 加 `surge`（注意系统库） - 有商业软件 → 按 3.4 节配置路径，用 `powermcp doctor` 验证

附录：资源索引

资源	链接
仓库主页	https://github.com/Power-Agent/PowerMCP
MCP 协议文档	https://modelcontextprotocol.io/introduction
MCP Python SDK	https://github.com/modelcontextprotocol/python-sdk
PowerIO	https://github.com/eigenergy/powerio
Easy SimAuto (PowerWorld 桥)	https://github.com/mzy2240/ESA
ANDES 文档	https://andes.readthedocs.io/
PyPSA	https://pypsa.org/
pandapower	https://github.com/e2n1EE/pandapower
故障评估演示视频	https://www.youtube.com/watch?v=MbF-SIBI4Ws
负荷增长评估演示视频	https://www.youtube.com/watch?v=euFUvhhV5dM
官方教程 PDF	仓库根目录 PowerMCP_Tutorial.pdf (注意：这是旧版手动配置方式，非推荐路径)

核心团队

Qian Zhang、Steven Black、Paulo Radatz、Andrea Pomarico、Muhy Eddin Za'ter、Luan Lopes dos Santos、Stephen Jenkins、Maanas Goel、Shen Wang、Drew Gray、Samuel Talkington

维护方：哈佛大学 SEAS 电力与人工智能倡议 (Power and AI Initiative, PAI)

本文档基于 PowerMCP 仓库源码与官方文档整理，生成时间：2026 年 9 月 19 日